**UDF - CENTRO UNIVERSITÁRIO DO DISTRITO FEDERAL**

CURSO DE CIÊNCIA DA COMPUTAÇÃO

DISCIPLINA – ANÁLISE E PROJETO DE SISTEMAS

# SISTEMA DF COLETA

## DOCUMENTAÇÃO DE NEGÓCIOS, REQUISITOS E MODELAGEM UML

**Alunos:**

Maycon de Sousa Palacio – RGM 44073593

Afonso Rafael Evangelista da Silva – RGM 43758401

André Luidhy Menezes Barbosa – RGM 43938451

Ruan Lopes Dourado – RGM 44469616

Filipe Augusto Izidro de Melo – RGM 44517441

Link da apresentação: [_____]

**Professor:**

Gabriel de Oliveira Alves

Brasília – 2026.2

**SUMÁRIO**

1 INTRODUÇÃO ..... 3

2 JUSTIFICATIVA ..... 3

3 OBJETIVOS ..... 4

3.1 Objetivo Geral ..... 4

3.2 Objetivos Específicos ..... 4

4 DESCRIÇÃO DO SISTEMA PROPOSTO ..... 5

5 REQUISITOS ..... 5

5.1 Requisitos Funcionais ..... 5

5.2 Requisitos Não Funcionais ..... 7

5.3 Regras de Negócio ..... 8

6 DIAGRAMAS UML ..... 9

6.1 Diagrama de Casos de Uso ..... 9

6.2 Diagrama de Classe ..... 17

6.3 Diagrama de Atividades ..... 20

6.4 Diagrama de Sequência ..... 23

6.5 Diagrama de Estado (ou Máquina de Estados) ..... 26

6.6 Diagrama de Componentes ..... 29

6.7 Diagrama de Objetos ..... 32

7 CONCLUSÃO ..... 34

8 REFERÊNCIAS ..... 34

## 1 INTRODUÇÃO

O descarte incorreto de resíduos sólidos é um problema crescente nos centros urbanos. No Distrito Federal, materiais recicláveis e itens que exigem logística reversa — como eletrônicos, pilhas, medicamentos vencidos, lâmpadas e óleo de cozinha — frequentemente são misturados ao lixo comum, contaminando o solo e sobrecarregando o sistema de limpeza pública. A informação sobre onde entregar cada tipo de resíduo existe, mas está dispersa em páginas oficiais, o que dificulta o descarte correto pelo cidadão.

Nesse contexto, este trabalho apresenta o DF Coleta, uma aplicação web que reúne, em um único lugar, os pontos de coleta de resíduos do Distrito Federal. O sistema permite ao cidadão consultar ecopontos, cooperativas de catadores, farmácias e estabelecimentos comerciais que recebem resíduos, filtrar por tipo de material, cidade e estabelecimento, visualizar os pontos no mapa e traçar a rota até eles. Ao administrador, oferece um painel para cadastrar, editar, ativar ou desativar pontos, gerenciar os tipos de resíduos e acompanhar indicadores, comentários, problemas relatados e sugestões da população.

O documento aqui apresentado registra o processo de análise e projeto do sistema: o levantamento de requisitos funcionais e não funcionais, as regras de negócio e a modelagem UML — por meio dos sete diagramas exigidos — que orientam a construção do protótipo funcional. A documentação de requisitos e a modelagem UML são etapas centrais do desenvolvimento de software porque transformam uma necessidade de negócio em uma especificação clara, verificável e compartilhada por todos os envolvidos, reduzindo retrabalho e aumentando a chance de o produto final atender ao que foi proposto.

## 2 JUSTIFICATIVA

O problema abordado é o descarte irregular de resíduos e a falta de informação centralizada sobre onde descartar corretamente no Distrito Federal. São afetados os moradores do DF, que não encontram facilmente um local de entrega; os catadores e as cooperativas, que dependem de material limpo e separado; os comerciantes, que precisam dar destinação correta aos seus resíduos; e o meio ambiente, que sofre com o descarte em terrenos vazios, córregos e aterros.

O projeto está vinculado aos Objetivos de Desenvolvimento Sustentável da ONU, especificamente ao ODS 11 — Cidades e Comunidades Sustentáveis, meta 11.6, que trata da redução do impacto ambiental negativo das cidades, inclusive na gestão de resíduos — e ao ODS 12 — Consumo e Produção Responsáveis, meta 12.5, relativa à prevenção, redução, reciclagem e reutilização. O DF Coleta contribui ao centralizar os pontos de coleta, os tipos de resíduo, os horários de funcionamento e os contatos, permitindo que o cidadão saiba o que, onde e quando descartar, e abrindo canais para sugerir novos pontos e relatar problemas.

Como resultados esperados, destacam-se: o aumento do descarte correto e da recuperação de materiais recicláveis; a redução dos pontos de descarte irregular; o fortalecimento das cooperativas, que recebem material de melhor qualidade; e a geração de dados organizados que podem apoiar o poder público na identificação de regiões com pouca oferta de pontos. Trata-se, portanto, de um software como instrumento de política ambiental: ele não substitui a ação pública, mas entrega informação organizada, consultável e aberta à população.

## 3 OBJETIVOS

### 3.1 Objetivo Geral

Desenvolver a documentação de negócios, os requisitos e a modelagem UML de um sistema web para localização de pontos de coleta de resíduos no Distrito Federal, com um protótipo funcional coerente com o escopo proposto.

### 3.2 Objetivos Específicos

Identificar e descrever os requisitos funcionais e não funcionais do sistema, com base nas necessidades do cidadão e do administrador.

Elaborar os sete diagramas UML previstos — casos de uso, classes, atividades, sequência, estados, componentes e objetos — representando diferentes visões do sistema.

Produzir a documentação de negócios e de requisitos, incluindo as regras de negócio vinculadas a cada requisito.

Desenvolver um protótipo funcional que implemente as funcionalidades centrais do escopo, permitindo a validação da modelagem na prática.

## 4 DESCRIÇÃO DO SISTEMA PROPOSTO

O DF Coleta é uma aplicação web voltada a dois perfis de usuário. O cidadão — que não precisa de cadastro — consulta os pontos de coleta, pesquisa por nome ou endereço, filtra por resíduo, cidade e tipo de estabelecimento, ordena por proximidade, visualiza os pontos no mapa e acessa a página de detalhe, onde encontra endereço, telefone, horários de funcionamento, resíduos aceitos, comentários e a opção de traçar a rota no Google Maps. Ele também pode comentar, relatar um problema nas informações e sugerir novos pontos. O administrador acessa um painel protegido por login para cadastrar, editar, ativar ou desativar e excluir pontos, gerenciar horários e tipos de resíduo, moderar comentários, analisar problemas e sugestões e acompanhar indicadores do sistema.

A base de dados contém 409 pontos reais distribuídos em 30 regiões administrativas, classificados em cinco tipos — ecoponto, cooperativa, farmácia, comércio e escola — e associados a 10 tipos de resíduo. As fontes oficiais utilizadas foram o SLU/DF (Papa-Entulho), o mapa de Pontos de Entrega Voluntária da SEMA e o Catálogo do IPE/DF (cooperativas de catadores e polígonos das regiões administrativas).

Tecnologicamente, a interface é construída com HTML, CSS e JavaScript, com mapa em Leaflet e OpenStreetMap e ícones Lucide. A aplicação é uma API REST serverless em NestJS publicada na Vercel, e os dados são persistidos em um banco PostgreSQL no Supabase, com Row Level Security (RLS) e autenticação via Supabase Auth. O front-end estático e a API compartilham o mesmo domínio, e as rotas de escrita exigem token de autenticação validado no servidor.

## 5 REQUISITOS

Os requisitos foram levantados a partir das necessidades dos dois perfis de usuário do sistema e organizados em funcionais, não funcionais e regras de negócio, conforme o modelo da disciplina.

### 5.1 Requisitos Funcionais

Os requisitos funcionais descrevem as ações que o sistema deve executar, produzindo resultados tangíveis para os usuários.

RF01 – Visualizar informações do projeto: o sistema deve apresentar informações sobre o projeto e seu objetivo.
RF02 – Visualizar tipos de resíduos: o sistema deve apresentar os tipos de resíduos aceitos nos pontos de coleta.
RF03 – Listar pontos de coleta: o sistema deve apresentar uma lista com os pontos de coleta cadastrados.
RF04 – Exibir pontos no mapa: o sistema deve apresentar os pontos de coleta em um mapa, conforme a localização do dispositivo.
RF05 – Pesquisar por endereço: o sistema deve permitir pesquisar pontos de coleta por endereço.
RF06 – Pesquisar por nome: o sistema deve permitir pesquisar pontos de coleta por nome.
RF07 – Filtrar por tipo de resíduo: o sistema deve permitir filtrar os pontos pelo tipo de resíduo.
RF08 – Filtrar por cidade: o sistema deve permitir filtrar os pontos por cidade.
RF09 – Filtrar por estabelecimento: o sistema deve permitir filtrar os pontos por tipo de estabelecimento.
RF10 – Visualizar dados do ponto: o sistema deve permitir visualizar os dados de um ponto de coleta.
RF11 – Apresentar endereço: o sistema deve apresentar o endereço do ponto de coleta.
RF12 – Apresentar horários: o sistema deve apresentar os horários de funcionamento quando o ponto estiver em um estabelecimento.
RF13 – Apresentar resíduos recebidos: o sistema deve apresentar os tipos de resíduos recebidos pelo ponto de coleta escolhido.
RF14 – Apresentar contato: o sistema deve apresentar telefone ou contato do ponto, quando disponível.
RF15 – Visualizar no Google Maps: o sistema deve permitir visualizar a localização do ponto no Google Maps.
RF16 – Ordenar por proximidade: o sistema deve permitir ordenar os pontos por proximidade do usuário.
RF17 – Acessar página de controle: o sistema deve disponibilizar uma página oculta para o administrador controlar o site.

RF18 – Cadastrar ponto de coleta: o sistema deve permitir ao administrador cadastrar pontos de coleta.
RF19 – Editar ponto de coleta: o sistema deve permitir ao administrador editar pontos de coleta.
RF20 – Excluir ponto de coleta: o sistema deve permitir ao administrador excluir pontos de coleta.
RF21 – Cadastrar tipo de resíduo: o sistema deve permitir ao administrador cadastrar tipos de resíduos.
RF22 – Editar tipo de resíduo: o sistema deve permitir ao administrador editar tipos de resíduos.
RF23 – Excluir tipo de resíduo: o sistema deve permitir ao administrador excluir tipos de resíduos.
RF24 – Cadastrar horários: o sistema deve permitir ao administrador cadastrar horários de funcionamento de um ponto.
RF25 – Ativar ou desativar ponto: o sistema deve permitir ao administrador ativar ou desativar pontos.
RF26 – Listar pontos cadastrados: o sistema deve permitir ao administrador visualizar a lista de pontos cadastrados.
RF27 – Comentar sobre um ponto: o sistema deve permitir que o usuário comente sobre um ponto por meio de um formulário.
RF28 – Relatar problema: o sistema deve permitir informar problemas nas informações de um ponto por meio de um formulário.
RF29 – Sugerir novo ponto: o sistema deve permitir ao usuário sugerir um novo ponto para análise do administrador.
RF30 – Visualizar indicadores: o sistema deve permitir ao administrador visualizar indicadores e dados do sistema em um painel.

### 5.2 Requisitos Não Funcionais

Os requisitos não funcionais determinam a qualidade e as restrições do sistema, como desempenho, segurança, usabilidade e conformidade.
RNF01 – Desempenho: o sistema deve responder às consultas de listagem e detalhe em até 2 segundos em condições normais de conexão.

RNF02 – Disponibilidade: o sistema deve permanecer disponível continuamente, sem janela de manutenção, por meio de hospedagem gerenciada.
RNF03 – Segurança: as rotas de escrita devem exigir token de autenticação válido, emitido pelo Supabase Auth, validado no servidor.
RNF04 – Usabilidade: a interface deve ser intuitiva e responsiva, sem rolagem horizontal, em telas de 360 px a 1440 px.
RNF05 – Compatibilidade: o sistema deve funcionar nos navegadores modernos (Chrome, Edge e Firefox) e em dispositivos móveis.
RNF06 – Confiabilidade: em caso de indisponibilidade da API, o sistema deve exibir um aviso e manter a interface utilizável.
RNF07 – Padrões: a documentação deve seguir a ABNT e o tratamento de dados deve respeitar a LGPD, sem expor e-mails em público.
RNF08 – Interoperabilidade: o sistema deve integrar-se ao OpenStreetMap (mapa) e ao Google Maps (rota) por interfaces públicas.

### 5.3 Regras de Negócio

As regras de negócio representam condições que regem o funcionamento do sistema e servem como critérios de aceitação. Cada regra está associada a um requisito.
RN01 – Somente pontos com status ATIVO são exibidos na consulta pública; os INATIVOS aparecem apenas no painel administrativo. (vinculada a RF25)
RN02 – Todo relato de problema nasce com status PENDENTE e só pode ter o andamento alterado pelo administrador no painel. (vinculada a RF28)
RN03 – Toda sugestão nasce PENDENTE e só se torna um ponto de coleta após a aprovação do administrador. (vinculada a RF29)
RN04 – Os filtros de resíduo, cidade e estabelecimento são combináveis; sem filtro, são listados todos os pontos ativos. (vinculada a RF07–RF09)
RN05 – Horários de funcionamento são opcionais e só são exibidos quando o ponto possui ao menos um registro. (vinculada a RF12)
RN06 – Telefone e contato são opcionais; campos sem valor não são exibidos na interface. (vinculada a RF14)
RN07 – Apenas o administrador autenticado pode criar, editar, ativar, desativar ou excluir pontos e resíduos. (vinculada a RF18–RF23, RF25)

RN08 – Um comentário pode ser curtido e pode receber uma resposta oficial da equipe, que fica sinalizada como verificada. (vinculada a RF27)
RN09 – A ordenação por proximidade usa a localização do dispositivo apenas quando o usuário autoriza o compartilhamento. (vinculada a RF16)
RN10 – Todo ponto precisa de coordenadas de latitude e longitude para aparecer no mapa. (vinculada a RF18)

## 6 DIAGRAMAS UML

Esta seção apresenta os sete diagramas previstos no modelo, cada um com seu resumo e sua imagem. As figuras são apresentadas em página própria, na orientação horizontal, para preservar a legibilidade.

### 6.1 Diagrama de Casos de Uso

#### 6.1.1 Resumo

O diagrama de casos de uso é uma representação visual da interação entre os atores e o sistema de software. Ele identifica os casos de uso — as funcionalidades que o sistema oferece — e mostra como os atores se relacionam com elas. Sua finalidade é entender os requisitos e a interação do usuário com o software a partir do ponto de vista de quem usa, sem entrar em detalhes de implementação.

No DF Coleta, o sistema é representado por uma fronteira rotulada com o nome da aplicação. Foram identificados dois atores humanos: o Cidadão, que consulta, filtra, comenta, relata problemas e sugere pontos, e o Administrador, que gerencia o conteúdo do painel. Há ainda o ator secundário API de Mapas, um sistema externo que fornece o mapa e recebe a rota.

O recorte inicial do sistema apresentava dezenas de funcionalidades. Como a rubrica da disciplina exige a descrição individual de todos os casos de uso do diagrama, a modelagem foi reduzida para doze casos essenciais, agrupados por ator e nomeados no infinitivo. Busca, filtros, ordenação, mapa e horários aparecem dentro dos fluxos dos casos, e não como elipses separadas.

Dois relacionamentos são usados. O relacionamento «include» liga o caso Gerenciar pontos de coleta ao caso Autenticar-se no painel, pois toda operação de

gestão exige login — sempre que o primeiro é executado, o segundo também é. O relacionamento «extend» liga Traçar rota no Google Maps a Visualizar detalhes do ponto, pois abrir a rota é uma ação opcional que o usuário pode ou não executar a partir do detalhe. A sincronização com uma API externa é prevista, mas ainda não implementada, e foi registrada como fluxo alternativo, não como caso de uso.

O diagrama, assim, cobre os requisitos funcionais RF01 a RF30 e serve de base para a descrição individual dos casos de uso e para os demais diagramas.

### 6.1.2 Desenho/imagem do diagrama

A Figura 1 apresenta o diagrama de casos de uso do DF Coleta.

Figura 1 – Diagrama de Casos de Uso




Fonte: elaborado pelos autores (2026).

### 6.1.3 Descrição dos Casos de Uso

Para garantir clareza e rastreabilidade, cada caso de uso apresentado no diagrama é descrito individualmente, com os sete campos do modelo: caso de uso, ator(es), descrição, pré-condição, fluxo principal, fluxo alternativo e pós-condições.

Quadro 1 – Descrição do caso de uso UC01

| **Campo** | **Descrição** |
|---|---|
| **Caso de Uso** | UC01 – Visualizar informações e tipos de resíduos |
| **Ator(es)** | Cidadão |
| **Descrição** | Permite ao cidadão conhecer o objetivo do projeto e os tipos de resíduos que os pontos de coleta aceitam. |
| **Pré-condição** | O sistema está disponível e acessível. |
| **Fluxo Principal** | 1. O caso de uso inicia quando o cidadão acessa a página inicial ou a página Sobre.<br>2. O sistema apresenta o objetivo do projeto e como o descarte correto funciona.<br>3. O sistema lista os tipos de resíduos aceitos, com nome e ícone.<br>4. O caso de uso é encerrado quando o cidadão navega para outra seção. |
| **Fluxo Alternativo** | a) Se a API estiver indisponível, o sistema exibe os tipos de resíduos de reserva e um aviso. |
| **Pós-condições** | O cidadão conhece o projeto e os resíduos aceitos pelos pontos. |

Fonte: elaborado pelos autores (2026).

Quadro 2 – Descrição do caso de uso UC02

| **Campo** | **Descrição** |
|---|---|
| **Caso de Uso** | UC02 – Consultar pontos de coleta |
| **Ator(es)** | Cidadão, API de Mapas |
| **Descrição** | Permite ao cidadão visualizar os pontos de coleta ativos, no mapa e em lista, e filtrá-los por resíduo, cidade e tipo de estabelecimento. |
| **Pré-condição** | O sistema está disponível. |
| **Fluxo Principal** | 1. O caso de uso inicia quando o cidadão acessa a página inicial ou a página de pontos.<br>2. O sistema carrega, em paralelo, os pontos ativos e os tipos de resíduo.<br>3. O sistema apresenta os pontos no mapa e em lista, com nome, endereço e resíduos.<br>4. O cidadão pode pesquisar por nome ou endereço e aplicar filtros de resíduo, cidade e estabelecimento.<br>5. O sistema aplica os filtros e ordena os pontos por proximidade quando a localização é autorizada.<br>6. O caso de uso é encerrado quando o cidadão sai da página ou abre um ponto. |
| **Fluxo Alternativo** | a) Se a API estiver indisponível, o sistema usa dados de reserva e avisa o usuário.<br>b) Se a busca não retornar resultados, o sistema exibe um estado vazio.<br>c) Se a geolocalização for negada, o sistema mantém a ordenação alfabética. |
| **Pós-condições** | O cidadão visualiza a lista e o mapa com os pontos filtrados conforme sua escolha. |

Fonte: elaborado pelos autores (2026).

Quadro 3 – Descrição do caso de uso UC03

| Campo | Descrição |
|---|---|
| **Caso de Uso** | UC03 – Visualizar detalhes do ponto |
| **Ator(es)** | Cidadão |
| **Descrição** | Permite ao cidadão ver os dados completos de um ponto de coleta. |
| **Pré-condição** | O cidadão selecionou um ponto na lista ou no mapa. |
| **Fluxo Principal** | 1. O caso de uso inicia quando o cidadão abre a página de detalhe de um ponto.<br>2. O sistema busca os dados do ponto e os comentários.<br>3. O sistema apresenta nome, tipo, endereço, cidade, telefone e status.<br>4. O sistema apresenta os horários de funcionamento, quando existirem.<br>5. O sistema apresenta os tipos de resíduo recebidos, com ícones.<br>6. O sistema apresenta o mapa com o marcador do ponto.<br>7. O caso de uso é encerrado quando o cidadão sai da página. |
| **Fluxo Alternativo** | a) Se o ponto não for encontrado, o sistema exibe uma mensagem de erro.<br>b) Se o ponto não tiver horários ou telefone, esses campos não são exibidos. |
| **Pós-condições** | O cidadão conhece o endereço, o contato, os horários e os resíduos do ponto. |

Fonte: elaborado pelos autores (2026).

Quadro 4 – Descrição do caso de uso UC04

| Campo | Descrição |
|---|---|
| **Caso de Uso** | UC04 – Traçar rota no Google Maps |
| **Ator(es)** | Cidadão, API de Mapas |
| **Descrição** | Permite ao cidadão abrir o trajeto até o ponto de coleta no Google Maps. |
| **Pré-condição** | O cidadão está na página de detalhe de um ponto. |
| **Fluxo Principal** | 1. O caso de uso inicia quando o cidadão escolhe abrir a rota.<br>2. O sistema monta a URL com as coordenadas do ponto.<br>3. O sistema abre o Google Maps em uma nova aba com o trajeto.<br>4. O caso de uso é encerrado. |
| **Fluxo Alternativo** | a) Se o navegador bloquear a nova aba, o cidadão pode copiar o link da rota. |
| **Pós-condições** | O cidadão tem o trajeto até o ponto aberto no Google Maps. |

Fonte: elaborado pelos autores (2026).

Quadro 5 – Descrição do caso de uso UC05

| Campo | Descrição |
|---|---|
| **Caso de Uso** | UC05 – Comentar sobre um ponto |
| **Ator(es)** | Cidadão |
| **Descrição** | Permite ao cidadão registrar uma avaliação ou observação sobre um ponto de coleta. |
| **Pré-condição** | O cidadão está na página de detalhe de um ponto. |
| **Fluxo Principal** | 1. O caso de uso inicia quando o cidadão preenche o formulário de comentário.<br>2. O cidadão informa nome, e-mail e o texto do comentário.<br>3. O sistema valida os campos obrigatórios.<br>4. O sistema grava o comentário vinculado ao ponto.<br>5. O sistema atualiza a lista de comentários da página.<br>6. O caso de uso é encerrado. |
| **Fluxo Alternativo** | a) Se os campos obrigatórios estiverem vazios, o sistema exibe uma mensagem e não grava.<br>b) Se a API estiver indisponível, o sistema avisa que o comentário não foi enviado. |
| **Pós-condições** | O comentário fica registrado e visível na página do ponto. |

Fonte: elaborado pelos autores (2026).

Quadro 6 – Descrição do caso de uso UC06

| Campo | Descrição |
|---|---|
| **Caso de Uso** | UC06 – Reportar problema |
| **Ator(es)** | Cidadão |
| **Descrição** | Permite ao cidadão informar que uma informação do ponto está incorreta ou desatualizada. |
| **Pré-condição** | O cidadão está na página de detalhe de um ponto. |
| **Fluxo Principal** | 1. O caso de uso inicia quando o cidadão abre o formulário de relato.<br>2. O cidadão informa o tipo de problema, a severidade e a descrição.<br>3. O sistema valida os campos obrigatórios.<br>4. O sistema grava o relato com status PENDENTE, vinculado ao ponto.<br>5. O sistema confirma o envio ao cidadão.<br>6. O caso de uso é encerrado. |
| **Fluxo Alternativo** | a) Se a descrição estiver vazia, o sistema exibe uma mensagem e não grava. |
| **Pós-condições** | O relato fica registrado e passa a constar na fila de moderação do painel. |

Fonte: elaborado pelos autores (2026).

Quadro 7 – Descrição do caso de uso UC07

| Campo | Descrição |
|---|---|
| **Caso de Uso** | UC07 – Sugerir novo ponto |
| **Ator(es)** | Cidadão |
| **Descrição** | Permite ao cidadão sugerir um novo ponto de coleta para análise do administrador. |
| **Pré-condição** | O sistema está disponível. |
| **Fluxo Principal** | 1. O caso de uso inicia quando o cidadão acessa a página de sugestão.<br>2. O cidadão informa nome do local, endereço, cidade e os resíduos aceitos.<br>3. O sistema valida os campos obrigatórios.<br>4. O sistema grava a sugestão com status PENDENTE.<br>5. O sistema confirma o envio ao cidadão.<br>6. O caso de uso é encerrado. |
| **Fluxo Alternativo** | a) Se os campos obrigatórios estiverem vazios, o sistema exibe uma mensagem e não grava. |
| **Pós-condições** | A sugestão fica registrada e passa a constar na fila de análise do painel. |

Fonte: elaborado pelos autores (2026).

Quadro 8 – Descrição do caso de uso UC08

| Campo | Descrição |
|---|---|
| **Caso de Uso** | UC08 – Autenticar-se no painel |
| **Ator(es)** | Administrador |
| **Descrição** | Permite ao administrador entrar na área restrita do sistema. |
| **Pré-condição** | O administrador possui credenciais válidas no Supabase Auth. |
| **Fluxo Principal** | 1. O caso de uso inicia quando o administrador acessa a página de login.<br>2. O administrador informa e-mail e senha.<br>3. O sistema valida as credenciais no Supabase Auth.<br>4. O sistema emite um token de sessão e o armazena no navegador.<br>5. O sistema redireciona o administrador para o painel.<br>6. O caso de uso é encerrado. |
| **Fluxo Alternativo** | a) Se as credenciais forem inválidas, o sistema exibe uma mensagem de erro. |

| | |
|---|---|
| | b) Se a sessão expirar, o sistema redireciona para o login. |
| **Pós-condições** | O administrador está autenticado e pode acessar as funções do painel. |

Fonte: elaborado pelos autores (2026).

Quadro 9 – Descrição do caso de uso UC09

| **Campo** | **Descrição** |
|---|---|
| **Caso de Uso** | UC09 – Gerenciar pontos de coleta |
| **Ator(es)** | Administrador |
| **Descrição** | Permite ao administrador cadastrar, editar, ativar ou desativar e excluir pontos, incluindo horários de funcionamento. |
| **Pré-condição** | O administrador está autenticado (UC08). |
| **Fluxo Principal** | 1. O caso de uso inicia quando o administrador abre a tela de pontos do painel.<br>2. O sistema lista todos os pontos, ativos e inativos.<br>3. O administrador escolhe cadastrar, editar, alterar o status ou excluir.<br>4. Para cadastrar ou editar, o administrador preenche nome, tipo, endereço, cidade, coordenadas, telefone, resíduos e horários.<br>5. O sistema valida e grava o ponto e seus vínculos no banco.<br>6. O sistema atualiza a lista e exibe uma confirmação.<br>7. O caso de uso é encerrado. |
| **Fluxo Alternativo** | a) Se um campo obrigatório estiver vazio, o sistema exibe uma mensagem e não grava.<br>b) Ao excluir, o sistema pede confirmação antes de remover o registro.<br>c) A sincronização com uma API externa de pontos é um fluxo alternativo previsto, ainda não implementado. |
| **Pós-condições** | O ponto é cadastrado, editado, ativado, desativado ou excluído e a lista reflete a alteração. |

Fonte: elaborado pelos autores (2026).

Quadro 10 – Descrição do caso de uso UC10

| **Campo** | **Descrição** |
|---|---|
| **Caso de Uso** | UC10 – Gerenciar tipos de resíduos |
| **Ator(es)** | Administrador |
| **Descrição** | Permite ao administrador cadastrar, editar e excluir os tipos de resíduos aceitos. |
| **Pré-condição** | O administrador está autenticado (UC08). |
| **Fluxo Principal** | 1. O caso de uso inicia quando o administrador abre a tela de resíduos do painel.<br>2. O sistema lista os tipos de resíduo cadastrados.<br>3. O administrador escolhe cadastrar, editar ou excluir.<br>4. O administrador informa nome, ícone e descrição do resíduo.<br>5. O sistema valida e grava o resíduo.<br>6. O sistema atualiza a lista e exibe uma confirmação.<br>7. O caso de uso é encerrado. |
| **Fluxo Alternativo** | a) Se o nome já existir, o sistema exibe uma mensagem e não grava.<br>b) Ao excluir, o sistema pede confirmação. |
| **Pós-condições** | O resíduo é cadastrado, editado ou excluído e a lista reflete a alteração. |

Fonte: elaborado pelos autores (2026).

Quadro 11 – Descrição do caso de uso UC11

| **Campo** | **Descrição** |
|---|---|
| **Caso de Uso** | UC11 – Moderar contribuições |
| **Ator(es)** | Administrador |
| **Descrição** | Permite ao administrador gerenciar comentários, problemas relatados e sugestões enviados pela população. |

| | |
|---|---|
| **Pré-condição** | O administrador está autenticado (UC08). |
| **Fluxo Principal** | 1. O caso de uso inicia quando o administrador abre as telas de comentários, problemas ou sugestões.<br>2. O sistema lista as contribuições recebidas.<br>3. O administrador pode excluir um comentário, responder oficialmente ou alterar o andamento de um problema.<br>4. Para sugestões, o administrador pode aprovar ou rejeitar.<br>5. Ao aprovar uma sugestão, o sistema cria um novo ponto de coleta com os dados informados.<br>6. O sistema persiste a decisão e atualiza a lista.<br>7. O caso de uso é encerrado. |
| **Fluxo Alternativo** | a) Se a sugestão não tiver os dados mínimos, o sistema avisa antes de criar o ponto. |
| **Pós-condições** | A contribuição é moderada e a decisão fica registrada no banco. |

Fonte: elaborado pelos autores (2026).

Quadro 12 – Descrição do caso de uso UC12

| **Campo** | **Descrição** |
|---|---|
| **Caso de Uso** | UC12 – Acompanhar indicadores |
| **Ator(es)** | Administrador |
| **Descrição** | Permite ao administrador visualizar indicadores e dados do sistema em um painel. |
| **Pré-condição** | O administrador está autenticado (UC08). |
| **Fluxo Principal** | 1. O caso de uso inicia quando o administrador abre o dashboard.<br>2. O sistema calcula os indicadores a partir dos dados: total de pontos, ativos, resíduos, comentários, problemas e sugestões.<br>3. O sistema apresenta os indicadores em cartões e tabelas.<br>4. O caso de uso é encerrado. |
| **Fluxo Alternativo** | a) Se a API estiver indisponível, o sistema exibe um aviso e os indicadores ficam vazios. |
| **Pós-condições** | O administrador tem uma visão geral do conteúdo e do uso do sistema. |

Fonte: elaborado pelos autores (2026).

## 6.2 Diagrama de Classe

### 6.2.1 Resumo

O diagrama de classes representa a estrutura estática do sistema, descrevendo as classes, seus atributos e métodos e os relacionamentos entre elas. É a principal ponte entre o modelo conceitual e a persistência: cada classe do domínio corresponde a uma tabela do banco de dados PostgreSQL usado pelo Supabase.

A classe central é PontoColeta, com os atributos de identificação, localização (nome, tipo, endereço, cidade, latitude, longitude, telefone e status) e os métodos de exibição de detalhes, cálculo de distância e alteração de status. PontoColeta se relaciona com Residuo por uma associação N:N, resolvida na prática pela tabela ponto_residuos, e possui uma composição com HorarioFuncionamento, pois os horários só fazem sentido vinculados a um ponto.

As contribuições do cidadão foram generalizadas na classe abstrata Contribuicao, que concentra id, nomeAutor, emailAutor e data. Comentario, ProblemaRelato e SugestaoPonto herdam de Contribuicao e acrescentam seus campos específicos: o comentário guarda texto, curtidas e o vínculo de resposta; o relato de problema guarda tipo, severidade e status; e a sugestão guarda nome do local, endereço, cidade, texto e resíduos sugeridos. A autoassociação de Comentario modela a resposta a outro comentário.

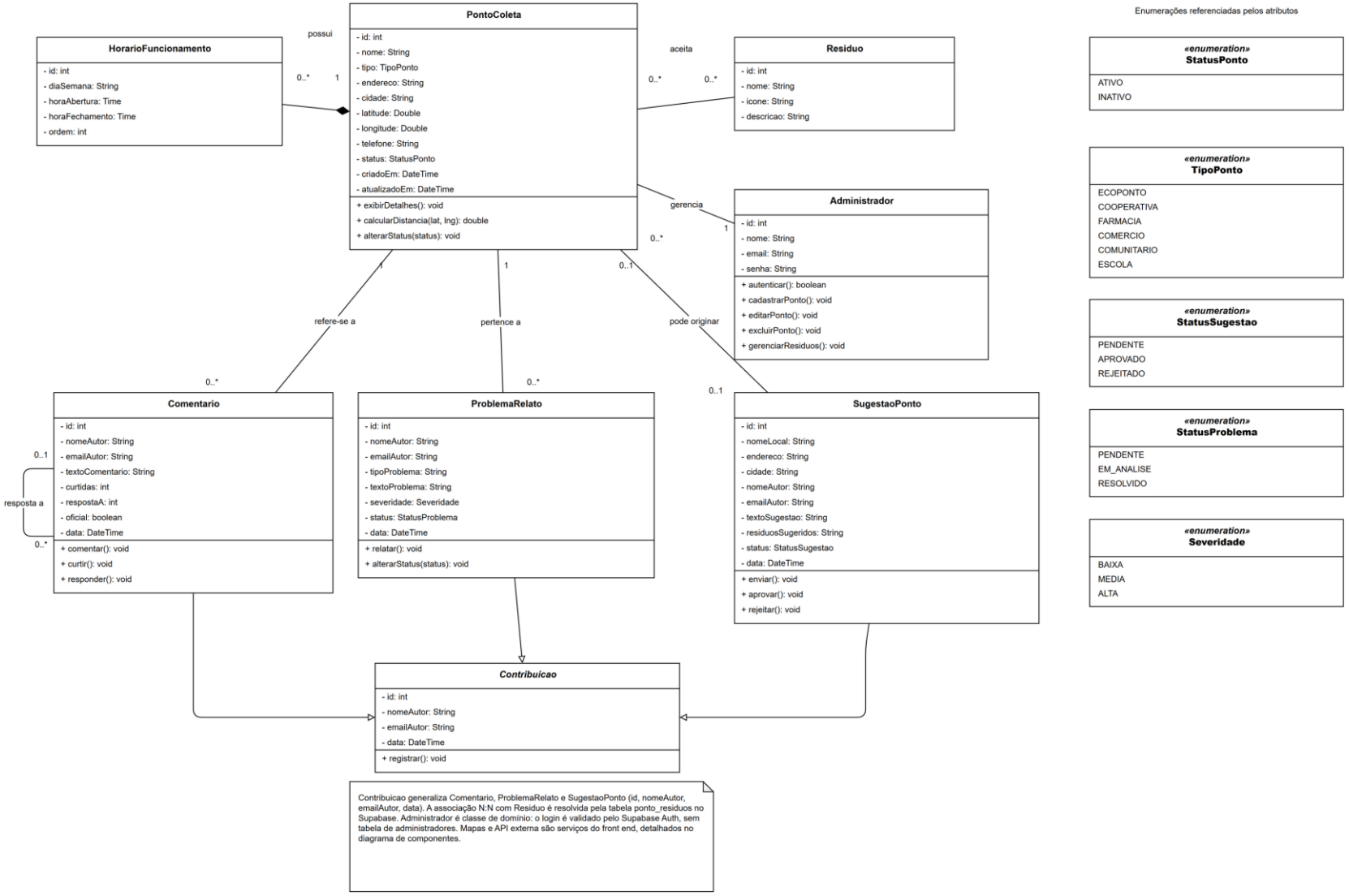
As enumerações StatusPonto, TipoPonto, StatusSugestao, StatusProblema e Severidade registram os valores aceitos pelo banco, garantindo consistência com as restrições reais das tabelas. A classe Administrador é modelada como classe de domínio, embora a autenticação seja feita pelo Supabase Auth, sem tabela própria.

O diagrama cobre os casos de uso UC02 a UC07, UC09 e UC11 e todas as tabelas do Supabase, com as multiplicidades lidas diretamente do modelo de dados.

### 6.2.2 Desenho/imagem do diagrama

A Figura 2 apresenta o diagrama de classe do DF Coleta.

Figura 2 – Diagrama de Classe




Fonte: elaborado pelos autores (2026).

## 6.3 Diagrama de Atividades

### 6.3.1 Resumo

O diagrama de atividades representa o fluxo de trabalho passo a passo de um processo, mostrando a ordem das ações, as decisões e os pontos em que o fluxo se divide ou se reúne. Ele é usado para modelar a dinâmica de um caso de uso.

O fluxo escolhido foi a consulta pública: o cidadão acessa a home, o navegador prepara as requisições e, em paralelo, a API consulta os pontos ativos e os tipos de resíduo — um fork que só se une (join) quando as duas respostas chegam. A decisão seguinte verifica se a API respondeu: em caso negativo, o sistema usa dados de reserva e avisa o usuário.

Com os dados carregados, o sistema renderiza o mapa, os destaques e os filtros. O cidadão pode então buscar, filtrar e ordenar os pontos. A decisão Há resultados? divide o fluxo: se não há correspondências, o sistema exibe um estado vazio; se há, o cidadão seleciona um ponto e abre a página de detalhe, que pede à API o ponto e os comentários.

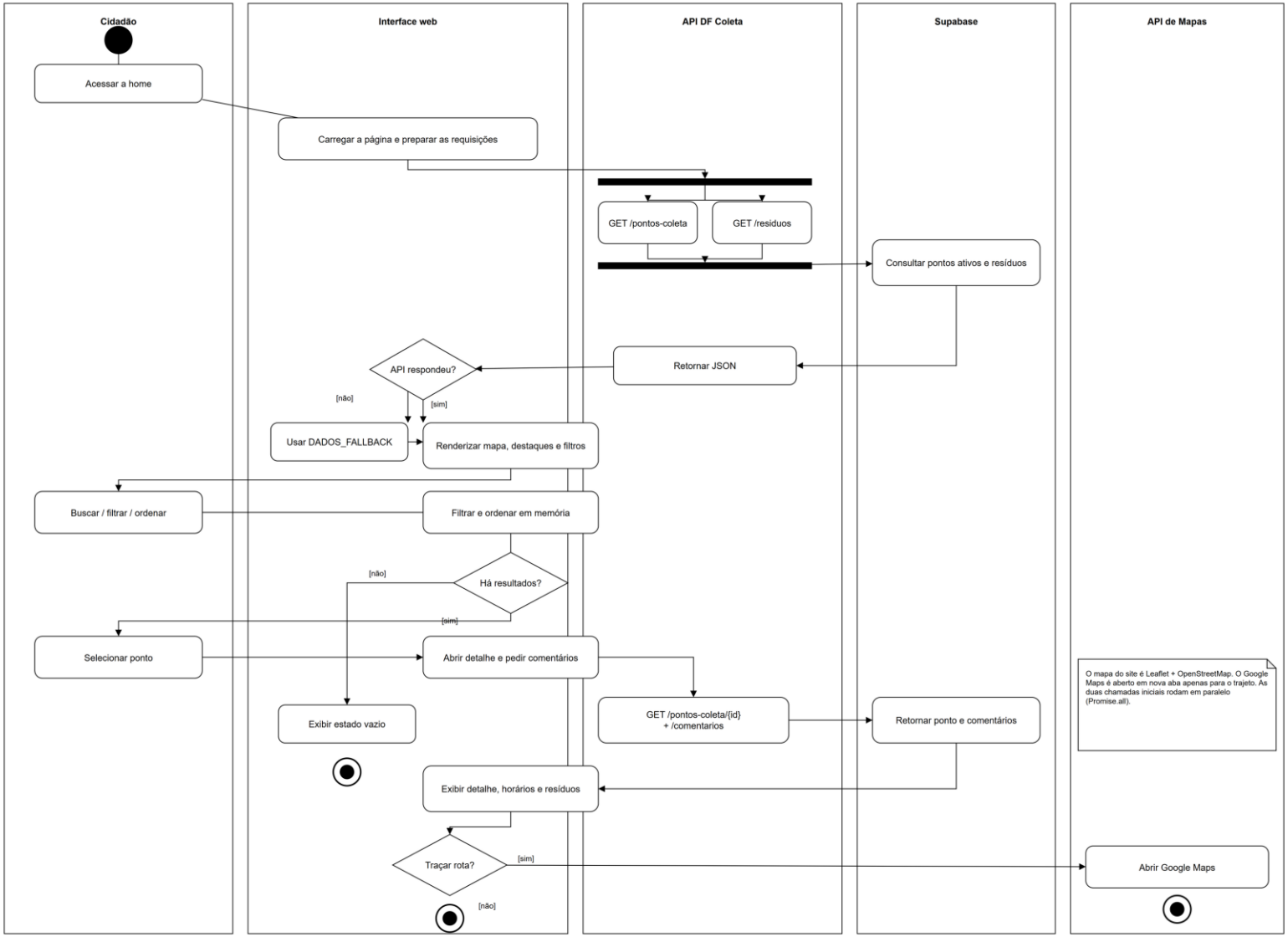
Na página de detalhe são exibidos endereço, horários e resíduos, e a última decisão pergunta se o usuário quer traçar a rota. Em caso positivo, o Google Maps é aberto em nova aba; em caso negativo, o fluxo termina. As raias (Cidadão, Interface web, API, Supabase e API de Mapas) agrupam as ações por responsável, e as guardas [sim] e [não] rotulam as saídas das decisões.

O diagrama cobre os casos de uso UC01 a UC04, incluindo os caminhos de exceção reais do sistema, como a indisponibilidade da API.

### 6.3.2 Desenho/imagem do diagrama

A Figura 3 apresenta o diagrama de atividades do DF Coleta.

Figura 3 – Diagrama de Atividades




Fonte: elaborado pelos autores (2026).

## 6.4 Diagrama de Sequência

### 6.4.1 Resumo

O diagrama de sequência detalha a troca de mensagens entre objetos ao longo do tempo, mostrando quem chama quem e em que ordem. Cada participante tem uma linha de vida tracejada, e as mensagens são setas horizontais lidas de cima para baixo.

O cenário modelado é o cadastro e a edição de um ponto de coleta no painel administrativo. Participam o Administrador, a tela de pontos, a API e o Supabase. O fluxo começa quando o administrador abre o painel: a tela verifica a sessão e carrega os resíduos e a lista completa de pontos, que o Supabase devolve à interface.

Ao clicar em Novo ponto ou Editar, a tela abre o modal, o administrador preenche e confirma, e a tela valida os campos. Se a validação falhar, um fragmento opt desvia para a exibição de um toast de erro, sem enviar nada ao servidor. Se passar, o fragmento alt escolhe entre POST, para um novo ponto, e PUT, para editar um existente.

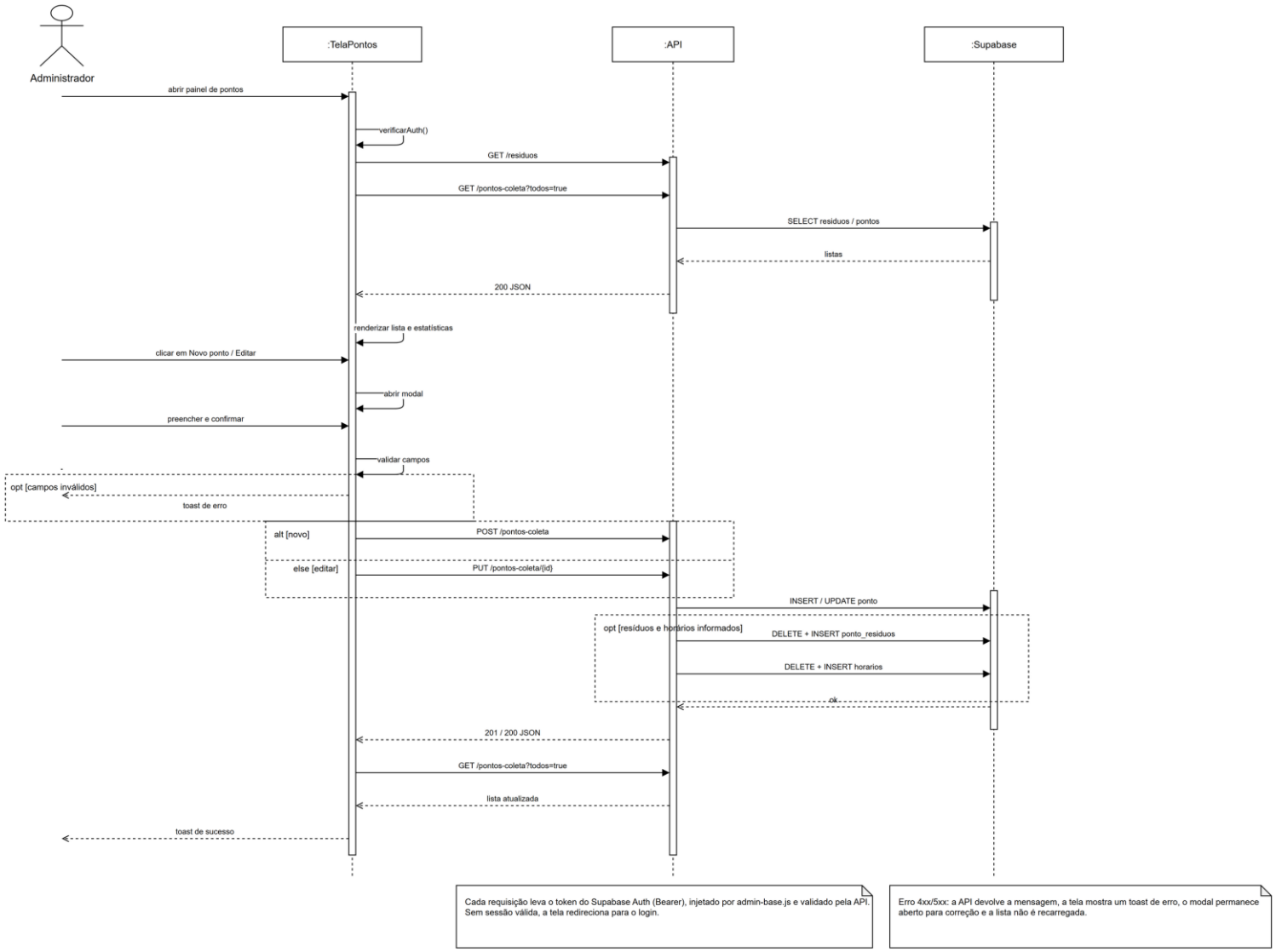
No servidor, a API grava o ponto e sincroniza os vínculos: apaga e reinsere os registros de ponto_residuos e de horarios_funcionamento, conforme o conteúdo informado. A resposta de sucesso retorna à tela, que recarrega a lista e exibe um toast de confirmação. Em caso de erro 4xx ou 5xx, a API devolve a mensagem e o modal permanece aberto para correção.

Cada requisição leva o token do Supabase Auth no cabeçalho, validado pela API antes de qualquer escrita. O diagrama cobre os casos de uso UC08 e UC09.

### 6.4.2 Desenho/imagem do diagrama

A Figura 4 apresenta o diagrama de sequência do DF Coleta.

Figura 4 – Diagrama de Sequência




Fonte: elaborado pelos autores (2026).

## 6.5 Diagrama de Estado (ou Máquina de Estados)

### 6.5.1 Resumo

O diagrama de estados (ou máquina de estados) descreve o ciclo de vida de um objeto, mostrando os estados possíveis e os eventos que provocam as transições entre eles. É útil para objetos cujo comportamento muda conforme a situação.

O objeto modelado é a SugestaoPonto, a sugestão de novo ponto enviada pelo cidadão. O ciclo começa quando o usuário envia a sugestão com dados válidos: o objeto entra no estado PENDENTE, que registra a data e exibe o item na fila de moderação.

Do estado PENDENTE partem duas transições, ambas dependentes da decisão do administrador: aprovar(), que leva ao estado APROVADO, e rejeitar(), que leva ao estado REJEITADO. Ao entrar em APROVADO, o sistema cria um novo PontoColeta com os dados da sugestão. O estado REJEITADO mantém o histórico e permite reativar(), que devolve a sugestão ao estado PENDENTE para nova análise.

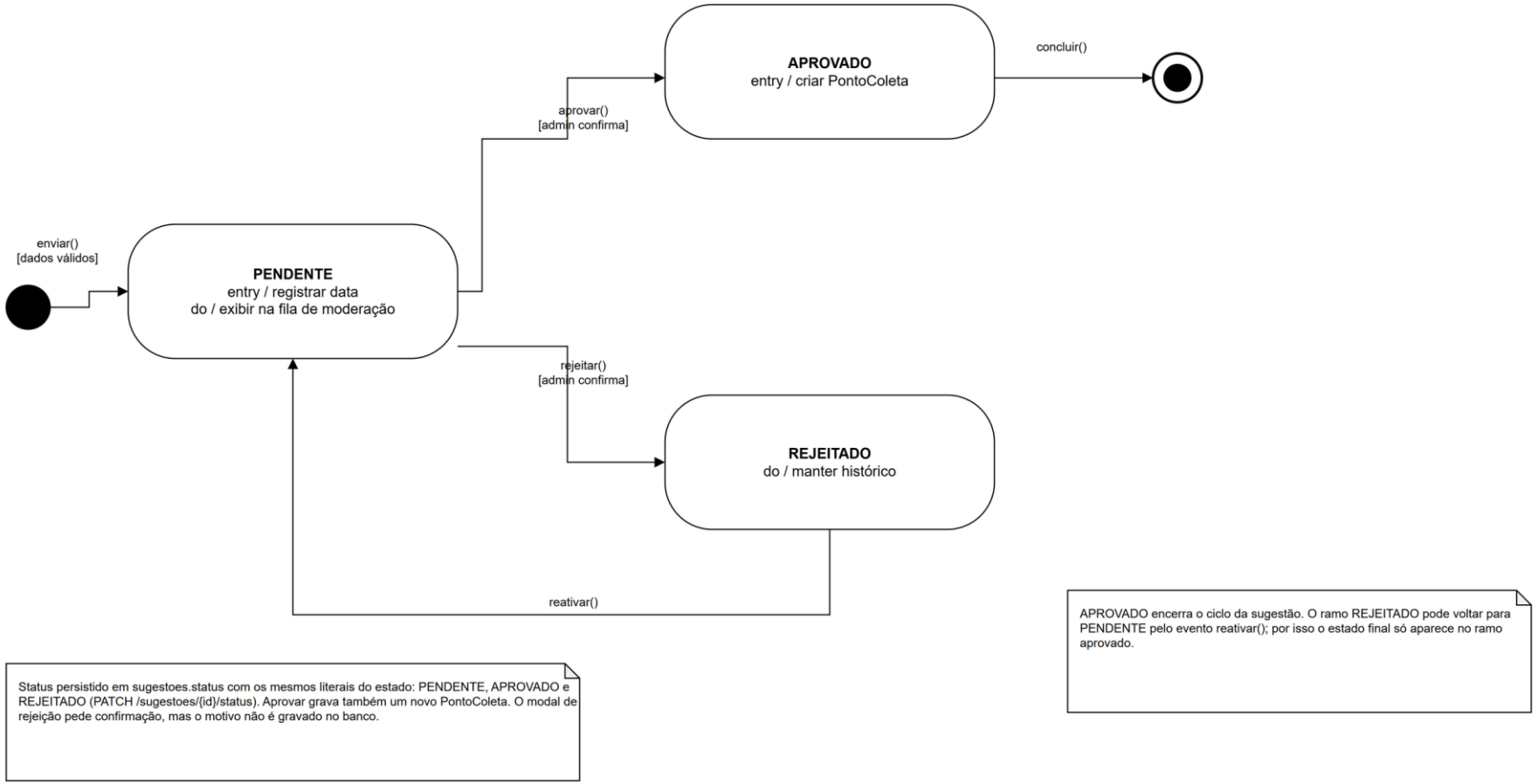
O estado final só aparece no ramo aprovado, pois uma sugestão rejeitada pode sempre ser reativada. Os literais dos estados — PENDENTE, APROVADO e REJEITADO — são os mesmos valores persistidos na coluna status da tabela sugestoes, gravados por meio de requisições PATCH no painel.

O diagrama cobre os casos de uso UC07 (enviar sugestão) e UC11 (moderar contribuições) e valida, com o ciclo real do sistema, os atributos e as multiplicidades da classe SugestaoPonto.

### 6.5.2 Desenho/imagem do diagrama

A Figura 5 apresenta o diagrama de estado (ou máquina de estados) do DF Coleta.

Figura 5 – Diagrama de Estado (ou Máquina de Estados)




Fonte: elaborado pelos autores (2026).

## 6.6 Diagrama de Componentes

### 6.6.1 Resumo

O diagrama de componentes representa a arquitetura lógica do sistema em módulos, mostrando os componentes, as interfaces que eles oferecem e as dependências entre eles. É a visão que liga a modelagem ao código realmente implementado.

O sistema é organizado em três camadas. A camada de apresentação contém as páginas públicas, o painel administrativo, o cliente HTTP api.js e o mapa.js. A camada de aplicação contém a API DF Coleta, uma função serverless em NestJS publicada na Vercel, que expõe as interfaces IPontosColeta, IContribuicoes e IIndicadores. A camada de dados contém o Supabase Postgres, com as tabelas, a view pontos_coleta_completo e as políticas de RLS.

Os serviços externos são o OpenStreetMap, de onde vêm os tiles do mapa, e o Google Maps, usado apenas para abrir o trajeto em nova aba. A integração com uma API externa de pontos, que permitiria sincronizar dados em lote, é prevista e aparece destacada, pois hoje a base oficial é importada manualmente para o Supabase.

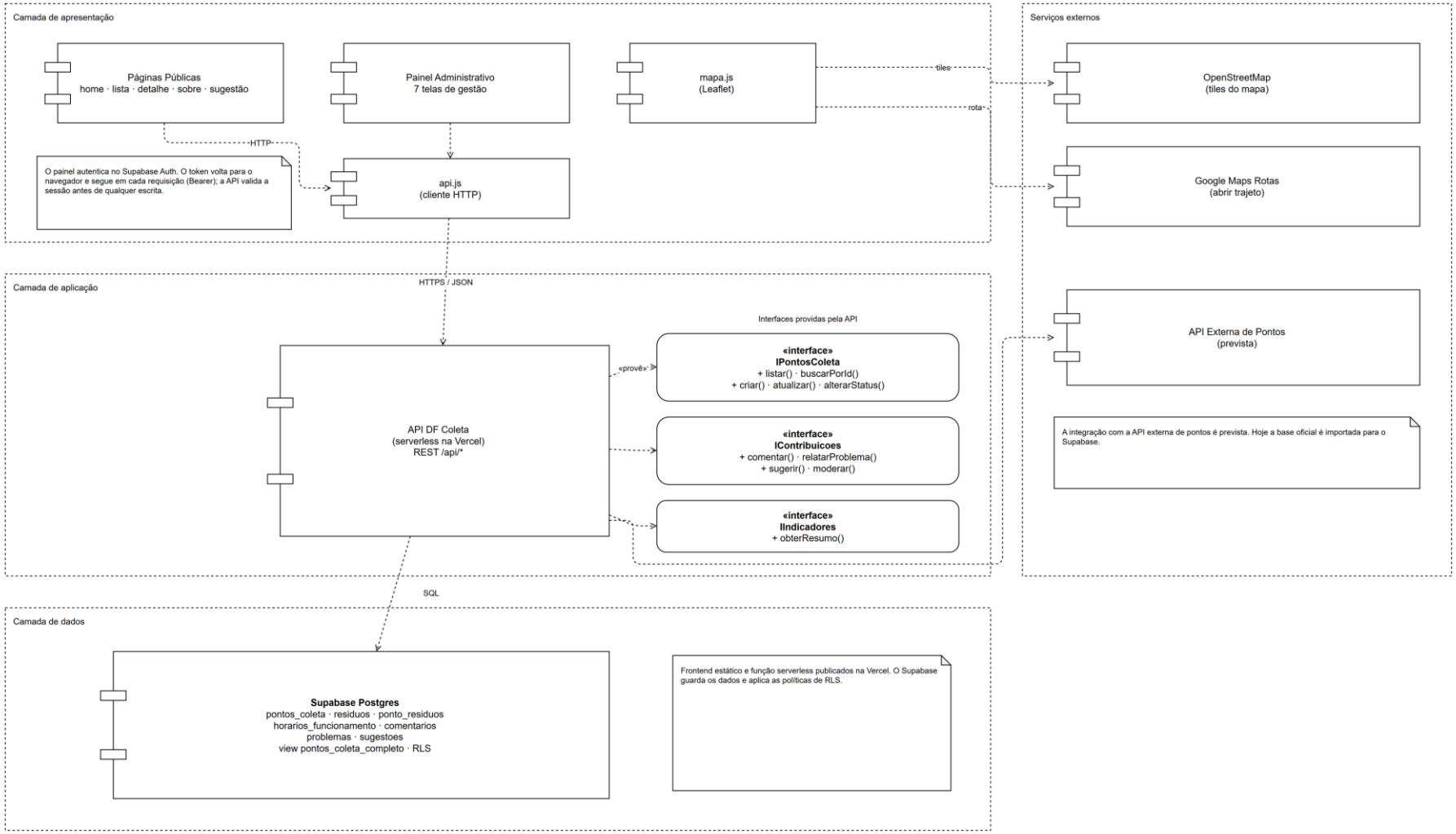
O fluxo de uma requisição mostra a dependência em camadas: as páginas chamam api.js, que fala HTTPS/JSON com a API, que por sua vez consulta o banco por SQL. O painel administrativo autentica no Supabase Auth, e o token segue em cada requisição de escrita. O front-end é estático, o que simplifica a hospedagem, e a API é serverless, o que elimina a gestão de servidor.

O diagrama oferece a visão de arquitetura de todos os casos de uso e destaca, como limitação, que a autenticação é delegada ao Supabase e a sincronização externa ainda não foi construída.

### 6.6.2 Desenho/imagem do diagrama

A Figura 6 apresenta o diagrama de componentes do DF Coleta.

Figura 6 – Diagrama de Componentes




Fonte: elaborado pelos autores (2026).

## 6.7 Diagrama de Objetos

### 6.7.1 Resumo

O diagrama de objetos é uma fotografia do sistema em um instante: ele mostra instâncias das classes, com valores concretos nos atributos, e os links entre elas. É usado para validar, na prática, as multiplicidades definidas no diagrama de classes.

O exemplo usa dados reais consultados na base em 02/11/2026. O objeto p401:PontoColeta representa a cooperativa ACAPAS, situada na SGON Quadra 05 Lote 23, Plano Piloto, com latitude -15.771496, longitude -47.913668, telefone (61) 3223-1400 e status ATIVO. O ponto está ligado a quatro objetos Residuo — Metal, Papel e Papelão, Plástico e Vidro — e a dois objetos HorarioFuncionamento, segunda e sábado, das 08:00 às 17:00, confirmando a associação N:N com os resíduos e a composição com os horários.

O link gerencia liga um objeto Administrador ao ponto, ilustrando a associação 1 para 0..* do diagrama de classes, e o link recebe liga um Comentario ao ponto, ilustrando 1 para 0..*. Os valores tornam tangível a diferença entre classe (um molde abstrato) e objeto (uma ocorrência real).

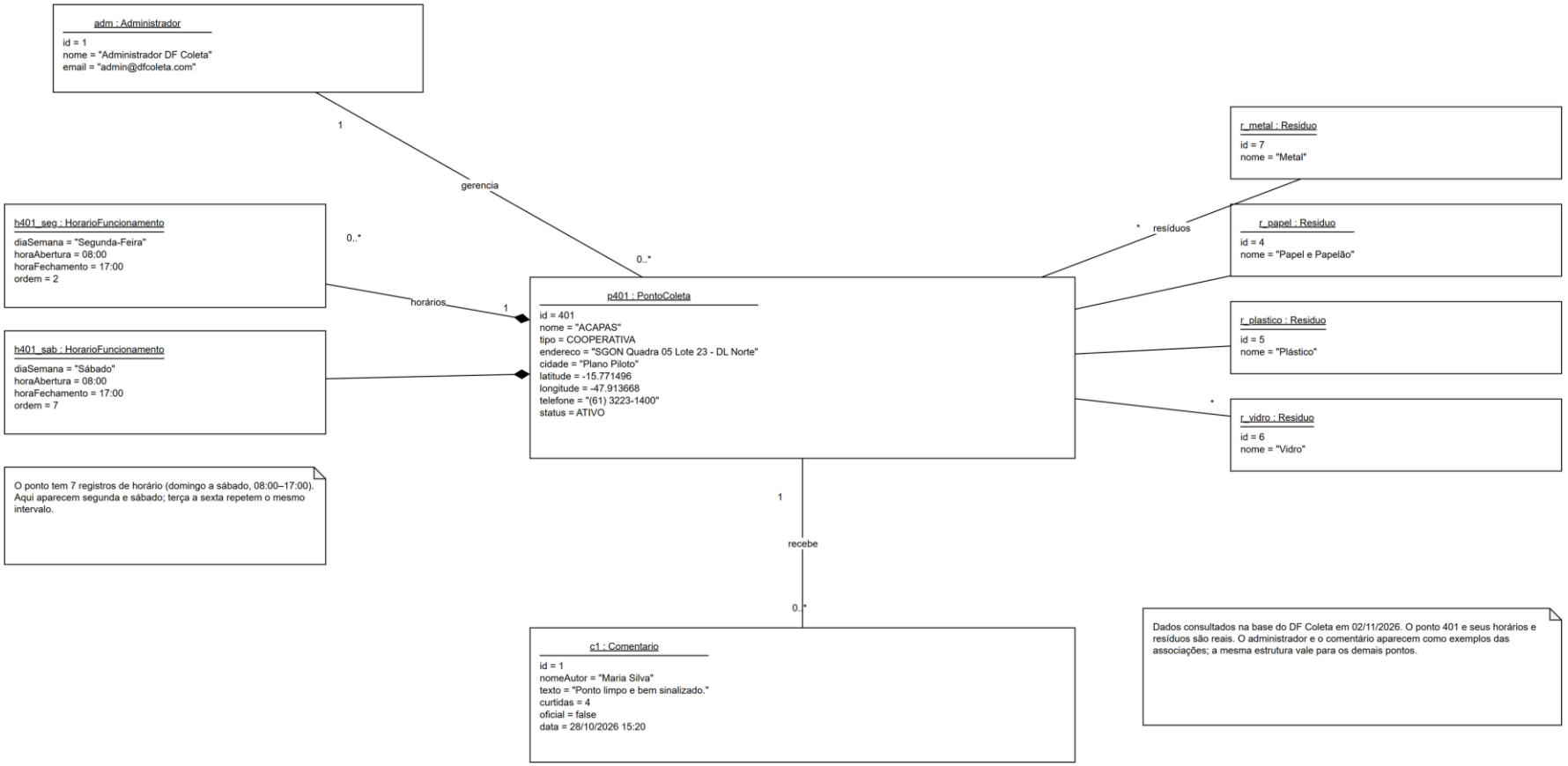
O exemplo prova que as multiplicidades modeladas são consistentes com a base: um ponto aceita vários resíduos, possui vários horários e recebe vários comentários. O administrador e o comentário aparecem como exemplos das associações, pois a mesma estrutura se repete para os demais pontos.

O diagrama cobre os casos de uso UC02, UC03, UC09 e UC11 e complementa o diagrama de classes com uma instância real do domínio.

### 6.7.2 Desenho/imagem do diagrama

A Figura 7 apresenta o diagrama de objetos do DF Coleta.

Figura 7 – Diagrama de Objetos




Fonte: elaborado pelos autores (2026).

## 7 CONCLUSÃO

O desenvolvimento do DF Coleta permitiu aplicar, em um problema real — o descarte correto de resíduos no Distrito Federal — as técnicas de análise e projeto de sistemas estudadas em aula. O levantamento de requisitos transformou uma necessidade difusa em 30 requisitos funcionais, oito não funcionais e dez regras de negócio verificáveis, e a modelagem UML ofereceu visões complementares do sistema: os casos de uso organizaram as funcionalidades em torno dos atores, as classes estruturaram o domínio e a persistência, os diagramas de atividades e de sequência detalharam os fluxos principais, o diagrama de estados modelou o ciclo de vida das sugestões e os diagramas de componentes e objetos conectaram a arquitetura ao código.

A principal dificuldade enfrentada foi manter a coerência entre a modelagem e o protótipo: à medida que o sistema foi implementado, parte da modelagem inicial deixou de corresponder ao que o código realmente fazia. A revisão item a item — comparando cada classe, atributo e relacionamento com o banco de dados e cada caso de uso com as telas — foi decisiva para eliminar inconsistências, como estados que não existiam e métodos não implementados. A experiência reforçou que a modelagem é um artefato vivo, que só tem valor quando permanece fiel ao sistema que descreve.

Conclui-se, assim, que o projeto atingiu os objetivos propostos: entregou uma solução delimitada, coerente e demonstrável, documentada segundo o modelo da disciplina, e demonstrou na prática como os diagramas UML apoiam a comunicação, a análise e a implementação de um sistema de software.

## 8 REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro: ABNT, 2011.

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. UML: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2012.

NAÇÕES UNIDAS BRASIL. Objetivos de Desenvolvimento Sustentável. Disponível em: https://brasil.un.org/pt-br/sdgs. Acesso em: 3 nov. 2026.

OPENSTREETMAP. OpenStreetMap. Disponível em: https://www.openstreetmap.org. Acesso em: 3 nov. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

SERVIÇO DE LIMPEZA URBANA DO DISTRITO FEDERAL. Papa-Entulho. Disponível em: https://www.slu.df.gov.br/papa-entulho. Acesso em: 2 nov. 2026.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

SUPABASE. Documentação. Disponível em: https://supabase.com/docs. Acesso em: 2 nov. 2026.

## FICHA DE AUTOAVALIAÇÃO DO GRUPO

A ficha de autoavaliação deve ser única por grupo e inserida na última página do trabalho. O preenchimento deve ser feito em conjunto, de forma justa e responsável, registrando o nível de participação de cada integrante. Cada critério deve ser avaliado de 0 a 5, em que 0 significa ausência de contribuição e 5, contribuição máxima.

| Integrante | Discussões | Conteúdo escrito | Diagramas UML | Organização | Apresentação |
|---|---|---|---|---|---|
| **Maycon de Sousa Palacio – RGM 44073593** | | | | | |
| **Afonso Rafael Evangelista da Silva – RGM 43758401** | | | | | |
| **André Luidhy Menezes Barbosa – RGM 43938451** | | | | | |
| **Ruan Lopes Dourado – RGM 44469616** | | | | | |
| **Filipe Augusto Izidro de Melo – RGM 44517441** | | | | | |

Fonte: elaborado pelos autores (2026). Critérios avaliados de 0 a 5: participação nas discussões, produção de conteúdo escrito, contribuição nos diagramas UML, colaboração na organização/formatação e participação na apresentação.